

Dobra

Day 2 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“do not contain \$\$\$ sequential vowels”*
- ▶ *“contain at least one letter”*

Programming interpretation:

- ▶ Underscores are unknown letters that must be substituted.
- ▶ The recursive state needs only the position, consecutive vowel count, consecutive consonant count, and whether L appeared.
- ▶ Group choices into vowel, consonant-not-L, and L instead of trying all 26 letters separately.

Textbook connection: CP4 3.2.2 Recursive Complete Search

Core technique: Grouped recursive backtracking

- ▶ Underscores can become vowels, consonants other than L, or the letter L.
- ▶ A valid word has no three consecutive vowels or consonants.
- ▶ A valid word must contain at least one L.

- ▶ Trying all 26 letters per underscore is wasteful.
- ▶ The rules depend only on vowel/consonant category and whether L has appeared.
- ▶ Grouping choices gives the same count with far fewer branches.

- ▶ Define `backtrack(pos, vowels, consonants, has_l)`.
- ▶ Prune immediately if `vowels >= 3` or `consonants >= 3`.
- ▶ At the end, return 1 only if `has_l` is true.
- ▶ For an underscore, branch into 5 vowels, 20 non-L consonants, and 1 L.
- ▶ For fixed letters, update the appropriate consecutive count.

- ▶ The multiplier 5 counts all vowel replacements without listing them.
- ▶ The multiplier 20 counts consonants except L.
- ▶ The L branch sets `has_l` to true and counts as a consonant.

- ▶ Every concrete replacement belongs to exactly one of the three underscore groups.
- ▶ The state records all information needed for future validity checks.
- ▶ Invalid prefixes are pruned only after they already violate a problem rule, so no valid word is lost.

Complexity: $O(3^U)$ recursive branches after grouping, where U is the number of underscores; $O(L)$ recursion depth.

- ▶ Memoization is possible but not required for this version.
- ▶ The branch weights are counts of letters, not probabilities.

Source lines: `dobra.py:38-45`

```
import sys

word = sys.stdin.readline().strip()

VOWELS = "AEIOU"

def backtrack(pos, vowels, consonants, has_l):
```

What this chunk does

- ▶ The whole word is read once and shared by the recursive function.
- ▶ VOWELS defines the only letters that reset the consonant streak.
- ▶ The parameters describe everything needed to continue the search.

Source lines: dobra.py:46-54

```
if vowels >= 3 or consonants >= 3:
    return 0

if pos == len(word):
    return 1 if has_l else 0

char = word[pos]
```

What this chunk does

- ▶ Any branch with three vowels or consonants in a row is immediately invalid.
- ▶ At the end of the word, the branch counts only if an L has appeared.
- ▶ The current character determines which transition to apply next.

Source lines: `dobra.py:56-68`

```
if char == "_":  
    total = 0  
  
    total += 5 * backtrack(pos + 1, vowels + 1, 0, has_l)  
  
    total += 20 * backtrack(pos + 1, 0, consonants + 1, has_l)  
  
    total += backtrack(pos + 1, 0, consonants + 1, True)  
  
    return total
```

What this chunk does

- ▶ Five vowel choices have identical future behavior, so multiply that branch by 5.
- ▶ Twenty consonants other than L also share one transition.
- ▶ The single L branch turns on `has_l` and counts as a consonant.

Source lines: `dobra.py:70-79`

```
elif char in VOWELS:
    return backtrack(pos + 1, vowels + 1, 0, has_l)

else:
    return backtrack(pos + 1, 0, consonants + 1, has_l or char == "L")

print(backtrack(0, 0, 0, False))
```

What this chunk does

- ▶ A fixed vowel increments the vowel streak and resets consonants.
- ▶ A fixed consonant increments the consonant streak and resets vowels.
- ▶ The final print starts the recursion with no letters processed and no L seen.

Solution file: `practice_contests/day_2/dobra.py`

Problem: <https://open.kattis.com/problems/dobra>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.